SOFTWARE ENGINEERING REPORT

ESP32 IoT Network Monitoring and Traffic Control

Software Requirements and Development Lifecycle Report

26 October 2026 | Version 1.0

Implementation baseline and verification record

This report documents the prototype requirements, software development process, architecture, interfaces, verification evidence and remaining deployment work. It is based on the project source and recorded checks, rather than simulated performance claims.

Current conclusion: the software and gateway firmware are implemented and software checks pass. Physical ESP32 deployment and evidence of improved network performance remain outstanding.

Executive summary and report guide

The project is an experimental platform for observing mixed IoT traffic and investigating whether gateway measurements can support useful traffic control. Three ESP32 sensor nodes generate HTTP and UDP messages. Phone clients add browsing and video traffic. A dedicated fourth ESP32 acts as a Wi-Fi NAT gateway between these clients and the existing MiFi network. A Python Flask server receives sensor messages, serves application traffic and presents a monitoring dashboard.

The gateway exposes read-only SNMPv2c software-interface counters. Independent HTTP probes provide request round-trip times and failure outcomes. The data pipeline forms time windows, predicts degradation in the following window, and compares a decision-tree model against a threshold baseline. A controller can change a sensor's UDP sending interval through a token-protected endpoint. Mininet supports a separate controlled comparison of offered traffic rates.

The evidence establishes software behavior, not operational effectiveness. On 26 October 2026, 41 Python/native integration tests and seven dashboard interaction tests passed after the dummy-data cleanup. A successful ESP32 firmware build was recorded previously for the 4 MB ESP32 Dev Module. Physical flashing, RF behavior, live NAT routing and comparative performance results have not been demonstrated.

Report guide

Pages	Subject
3 to 6	Problem definition, functional requirements, quality requirements and resources
7 to 11	Architecture, data design, SNMP, prediction and control
12 to 14	SDLC process, implementation structure and verification
15 to 17	Deployment, research validation, maintenance and cleanup
18 to 19	Requirements traceability, references and terminology

Implemented means corresponding code exists. Software verified means an automated or native software check passed. Hardware pending means a physical-board or real-network acceptance check remains. Proposed means the capability or experiment is recommended but not demonstrated. These states are not equivalent.

1 Problem definition and scope

Problem statement

Sensor messaging, browsing and video streaming can compete for shared network resources. Endpoint observations may show delay or missing messages without explaining what the gateway is handling. The available MiFi does not provide an established SNMP interface for this project. A programmable ESP32 gateway supplies an accessible measurement point, while introducing a resource-constrained forwarding device whose own limits must be evaluated.

The research question is whether current endpoint and gateway observations can predict subsequent service degradation, and whether acting on that prediction improves sensor service compared with no control and a simple threshold controller. Degradation is an observed service outcome; predicting it does not establish that congestion is its cause.

Objectives

- Generate identifiable HTTP and UDP sensor workloads alongside competing application traffic.
- Record delivery observations, actual request round-trip times and explicit measurement failures.
- Collect two-interface gateway counters without requiring SNMP support from the MiFi.
- Build causal time-window datasets and evaluate prediction on separate capture sessions.
- Apply bounded sensor rate changes and record their effects and costs.
- Provide a usable dashboard, exports and repeatable software checks.

Stakeholders and responsibilities

The experiment operator configures the network, flashes boards, labels runs and executes comparisons. The software maintainer manages code, dependencies, credentials, tests and recovery. The researcher defines service limits and interprets evidence. A report reviewer needs traceable requirements and a distinction between implementation and measured results. These roles may be performed by the same person; no formal staffing is assumed.

Scope boundaries

The local prototype includes a Flask server, browser dashboard, sensor sketches, gateway firmware, SNMP collection, data processing, model training, interval control and Mininet experiments. It does not implement production network management, SNMPv3 on the custom gateway, a complete wireless simulator, cryptographic device identity or a proven optimization policy. Model training stays on the PC. Existing captures and historical models remain reference material.

2 Functional requirements

The requirements describe the implemented baseline and the evidence needed for acceptance. Physical acceptance remains a separate gate where stated.

ID	Requirement	Acceptance evidence
FR01	Generate HTTP and UDP sensor payloads with device identity, boot ID and independent transport sequences.	Sketch inspection; physical transmission pending.
FR02	Provide a downstream hotspot, DHCP range and upstream IPv4 NAT route.	Firmware build passes; board forwarding pending.
FR03	Expose read-only SNMP counter and discovery objects for two interfaces.	Native C object, walk and rejection tests pass; board polling pending.
FR04	Poll counters and derive rates while retaining resets, warmup and failures.	Counter and wire-client tests pass.
FR05	Persist observations and retain logical source plus original NAT peer.	HTTP/NAT integration and migration tests pass.
FR06	Estimate observed sequence gaps without treating duplicates as new deliveries.	Gap, duplicate and late-arrival tests pass.
FR07	Measure HTTP RTT and failures for direct or forwarded targets.	Mapping and pipeline checks pass; live probes pending.
FR08	Display status, traffic, RTT, delivery gaps, activity and counter availability.	Seven interaction checks and prior visual review pass.
FR09	Export observations, delivery summaries and stored measurements.	Flask endpoint checks pass; operator file review required.
FR10	Build current features and next-window labels without bridging missing windows.	Causal alignment, outage and missing-window checks pass.
FR11	Evaluate a classifier on held-out runs and report a threshold baseline.	Separate build/train CLI integration test passes.
FR12	Offer observe, threshold and model control with bounds, cooldown and logs.	Mocked action check passes; physical actuation pending.
FR13	Run repeated concurrent Mininet workloads with matched physical settings.	Launch/configuration checks pass; Linux run pending.
FR14	Generate matching gateway/sensor settings from private local configuration.	Configuration and subnet validation tests pass.

A successful compile does not satisfy live routing acceptance, and an interaction test does not establish network performance. Section 16 maps each requirement to source and verification evidence.

3 Quality requirements and constraints

ID	Quality requirement	Criterion and current interpretation
NFR01	Measurement integrity	Unknown RTT, failed polls and unsupported metrics stay missing. Jitter, inter-arrival and RTT remain distinct. Checks pass.
NFR02	Reproducibility	Record run IDs, workload settings, raw outputs and held-out sessions. Repeat comparable physical trials. Software support exists.
NFR03	Access restriction	Read-only gateway SNMP and collector ACL; token-protected sensor writes. Physical enforcement pending.
NFR04	Failure handling	Preserve failures, invalidate reset intervals and reject stale control input. Software verified; interruption trials pending.
NFR05	Resource discipline	Bound SNMP packet and binding counts. Measure heap and forwarding capacity during pilot. Parser bounds tested; load trials proposed.
NFR06	Data continuity	Preserve historical rows during migration; separate boots and transports. Grouping and migration checks pass.
NFR07	Operator usability	Meaningful units, unknown states, pause/resume, exports and responsive layout. Interaction and prior visual evidence available.
NFR08	Maintainability	Separate routing, collection, telemetry, training, control and presentation. Document setup and verification.
NFR09	Portability	PC server/collectors; pinned original-ESP32 build; real Mininet on Linux. Other boards need a new target.
NFR10	Research validity	Compare no control, threshold and model control on separate sessions; report cost and uncertainty. Experiments pending.

Operating constraints

The original ESP32 has limited memory and one shared Wi-Fi channel in AP plus station operation. Forwarding, SNMP and application traffic share its resources. Its counters cannot establish radio airtime or losses before the driver. MiFi and gateway subnets must differ. Keep the server upstream so sensor and phone traffic crosses the gateway. Reference [R1] describes the AP/station channel constraint.

SNMPv2c has no confidentiality; its ACL restricts access but does not encrypt packets. The Flask development server and application endpoints are intended for an isolated lab, not an Internet-facing service. Define performance and reliability targets before physical evaluation rather than inferring them from a successful demo.

4 Hardware and software requirements

Hardware baseline

Resource	Minimum or chosen baseline	Purpose
Gateway board	One original ESP32 Dev Module with 4 MB flash	Dedicated AP/station routing and SNMP
Sensor boards	Three Wi-Fi-capable ESP32 boards	HTTP plus UDP, HTTP-only and UDP-only roles
Server PC	Python-capable PC on upstream Wi-Fi	Flask, collection, logging and training
Existing MiFi	Upstream Wi-Fi; no SNMP requirement	Connect the PC and gateway station
Phone clients	One or two Wi-Fi browser clients	Browsing and sustained streaming traffic
Programming equipment	Data-capable USB cables and stable power	Flash and power four boards
Linux environment	Mininet, Open vSwitch and iperf3	Controlled emulation experiments

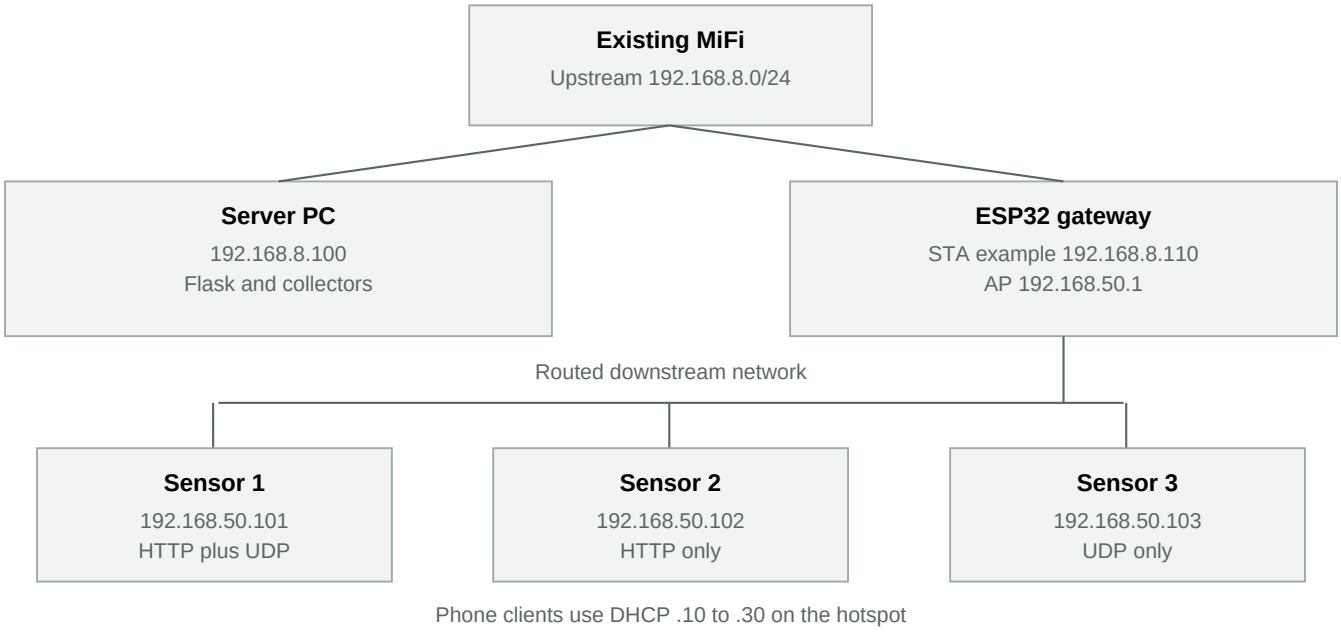
Software baseline

Software	Version or requirement	Use
Python	3.10 or later documented; 3.13.7 tested	Application, collectors and tests
Flask and Requests	Flask 3.x; Requests 2.x	HTTP server and endpoint probes
NumPy and pandas	NumPy 1.26 to below 3; pandas 2.2 to below 3	Features, windows and analysis
scikit-learn	1.5 to below 2	Training pipelines and evaluation
Matplotlib	3.8 to below 4	Stored analysis figures
Gateway build tools	PlatformIO 6.1.18; espressif32 6.10.0; ESP-IDF 5.4.0	Pinned gateway build
Sensor build tools	Arduino IDE and Espressif ESP32 board support	Build and flash sensor sketches
Browser and Chart.js	Modern browser; bundled Chart.js	Local dashboard without a CDN

The Python SNMPv2c backend needs no external SNMP utility. Net-SNMP is optional for other agents or discovery. SQLite, CSV, sockets and threading use the Python standard library. Native codec tests use Zig 0.14.1. Wireshark supports independent protocol inspection. Credentials, serial ports and actual network addresses are configured locally; no real password is included here.

5 System architecture and network design

The design separates the PC application, gateway measurement/forwarding functions and traffic sources. Example addresses document configuration, not an observed physical deployment.



Endpoint	Example address	Role
Upstream MiFi	192.168.8.1	Existing Wi-Fi network
Server PC	192.168.8.100	Flask TCP 5000; UDP receiver 12345
Gateway station	DHCP, for example 192.168.8.110	Upstream NAT address and SNMP
Gateway hotspot	192.168.50.1	Downstream gateway and DHCP
Sensors 1 to 3	192.168.50.101 to 103	Static sensor addresses
Phone clients	DHCP 192.168.50.10 to 30	Workload clients behind gateway

NAT translates downstream sources to the station address. The server maps a known payload device ID to its logical sensor IP only when the peer equals the configured gateway. It stores logical source and original peer separately. This is a lab mapping, not authenticated device identity. Phone requests are grouped by translated source.

Upstream TCP ports 8101, 8102 and 8103 forward to port 80 on the corresponding sensor. The PC probes each sensor through its forwarded URL. ICMP cannot use TCP port mappings, so gateway mode reports HTTP RTT and disables direct sensor ping sweeps. Reference [R2] documents the Espressif AP/station NAT example.

6 Data design and measurement definitions

The events table in server/telemetry.db records persistent delivery observations and supports CSV export. Dashboard counters are not the primary training record.

Field	Type	Meaning
timestamp	REAL	Server observation time in epoch seconds
run_id	TEXT	Explicit experiment session identifier
source	TEXT	Direct source or configured logical sensor IP
device	TEXT	Identifier supplied in the payload
protocol	TEXT	Application transport such as HTTP or UDP
bytes	INTEGER	Recorded application payload size
boot_id	TEXT	Device boot/session identifier
seq	INTEGER or null	Independent transport sequence
received	INTEGER	Accepted delivery or explicit application discard
peer_ip	TEXT	Original on-wire/NAT peer for audit

For one boot, device, source and transport, the expected observed span is last sequence minus first sequence plus one. Missing messages equal that span minus distinct received sequences. Gap percentage is 100 times missing divided by span. Duplicates do not improve delivery; late packets can repair gaps.

The calculation cannot infer missing prefixes, tails or completely unseen boots. It is not direct radio loss. Application discards are separately flagged and disabled by default using IOT_DROP_RATE=0.

Probe CSV rows contain time, run ID, logical source, successful HTTP duration, success flag and error class. Failures have missing RTT. SNMP CSV rows retain raw counters, rates and collection state. metrics.db stores stream and sensor metrics; JSON control logs store decisions and actions. Join records only with matching run IDs and compatible measurement definitions.

7 SNMP interface specification

The agent supports bounded SNMPv2c GET and GETNEXT on UDP 161. SET returns notWritable. SNMPv1, SNMPv3, GETBULK and traps are not implemented. Community and collector IPv4 ACL are configured locally. ifIndex 1 identifies the upstream station; ifIndex 2 identifies the hotspot.

Object	Instance or OID	Interpretation
sysUpTime	1.3.6.1.2.1.1.3.0	Boot uptime in hundredths of a second
ifHCInOctets	1.3.6.1.2.1.31.1.1.1.6.i	Bytes entering software interface
ifHCOutOctets	1.3.6.1.2.1.31.1.1.1.10.i	Bytes successfully handed to driver
ifHighSpeed	1.3.6.1.2.1.31.1.1.1.15.i	Zero because fixed capacity is not measured
ifCounterDiscontinuityTime	1.3.6.1.2.1.31.1.1.1.19.i	Counter installation/reset time
ifInDiscards	1.3.6.1.2.1.2.2.1.13.i	Input memory or buffer failures
ifOutDiscards	1.3.6.1.2.1.2.2.1.19.i	Output memory or buffer failures
ifInErrors	1.3.6.1.2.1.2.2.1.14.i	Other input callback errors
ifOutErrors	1.3.6.1.2.1.2.2.1.20.i	Other output callback errors
ifOperStatus	1.3.6.1.2.1.2.2.1.8.i	Software interface up/down

The required set has one shared uptime scalar and nine objects on each interface: 19 instances. Discovery and 32-bit compatibility objects extend the catalog to 38 instances. Limits are 1,400 bytes per packet, 24 variable bindings and 16 arcs per OID.

Bit rate is eight times octet-counter difference divided by elapsed monotonic seconds. Counter decreases, uptime rollover, discontinuity changes and failed polls invalidate the interval. The first sample is warmup. Utilization is missing when capacity is unknown; advertised PHY rate is not substituted. Counters include management traffic but do not expose airtime, retries, every internal NAT drop or packets lost before the driver. Zero errors cannot prove a loss-free wireless link.

8 Prediction pipeline and model evaluation

The default analysis unit is a five-second window per run and logical source. The builder preserves measured outage windows, excludes the final incomplete probe bucket and never bridges a missing future window. Define a warmup policy for the first partial window.

Feature	Definition
packet_rate	Received application messages divided by window seconds
byte_rate	Received application bytes divided by window seconds
packet_size_mean and std	Application message-size distribution
inter_arrival_mean_ms	Mean time between messages from the source
udp_fraction	Fraction of received messages using UDP
rtt_ms	Mean successful HTTP RTT in the window
probe_failure_pct	Percentage of failed HTTP probes
Optional SNMP inputs	Interface rates, discard/error rates and available utilization

SNMP samples join backward by completion time and run ID for one chosen agent/interface. Samples older than two window lengths are not reused. Failed polls clear features. Data arriving later therefore cannot populate an earlier feature window.

The target describes the immediately following measured window. Default degradation is mean successful HTTP RTT above 100 ms or probe failures at least 10 percent. These configurable values are research settings, not validated service-level guarantees.

The decision-tree classifier uses depth six, minimum five samples per leaf, balanced class weights and seed 42. Median imputation and missing indicators are fitted only on training runs. Complete selected runs are held out. Outputs include per-class precision/recall/F1, confusion matrix, session provenance and a threshold-persistence baseline. An optional tree regressor predicts next-window successful HTTP RTT when sufficient data exists.

Base and SNMP-augmented models use the same held-out rows. Reference [R3] supports training-only preprocessing. Historical packet-rule models remain available, but do not establish future degradation prediction. This report asserts no current model accuracy or physical performance improvement.

9 Control process and safeguards

Observe records conditions without intervention. Threshold uses current RTT/failure symptoms to choose a setting. Model predicts next-window degradation using a trained classifier. Intervention remains dry-run unless the operator supplies --apply and the matching control token.

Decision sequence

1. Retrieve recent observations and the probe log for the selected run.
2. Construct completed windows with the training duration and thresholds.
3. Reject missing/stale input and do not reprocess the same window.
4. Determine whether the protected source is degraded or predicted to degrade.
5. Limit the selected UDP source when needed; require sustained healthy observations before restoration.
6. Apply the bounded setting through a direct or forwarded endpoint when enabled, and log the result.

Parameter	Current setting
Normal UDP interval	1,100 ms by default
Limited UDP interval	3,000 ms by default
Firmware bounds	50 to 10,000 ms
Cooldown	15 seconds by default
Restoration	Three healthy windows
Sensor endpoint	HTTP POST /config with X-Control-Token
Stale or missing input	No new configuration action
Action record	JSON lines with run, strategy, decision and status

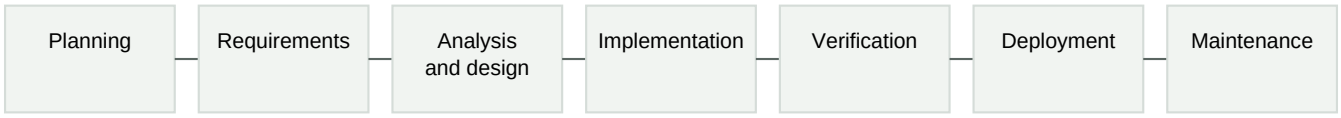
The protected source and rate-limited device may differ. A controller can protect sensor 1 while changing sensor 3 through gateway port 8103. Reducing offered load may improve service, but lowers sensor update frequency; that cost must be reported.

A sensor currently retains its last setting if the controller disappears. A device-side expiring lease is recommended. Load pickle models only from the operator's trusted workflow. An HTTP-trained model must not be applied uncritically to unrelated Mininet iperf features.

10 Software development lifecycle

Lifecycle approach

The work is best described as iterative prototyping with verification gates. This is a retrospective organization of implemented work, not a claim that formal Scrum ceremonies or a certified process were followed. Each increment addresses a capability or correctness issue, adds relevant checks and leaves physical acceptance explicit.



Iterate after verification or operational feedback

Phase	Work and output	Status
Planning	Define mixed traffic and control objectives; identify unavailable MiFi SNMP access.	Scope documented
Requirements	Define functional/quality needs, measurement semantics and acceptance criteria.	Baseline documented
Analysis and design	Separate outcomes from traffic features; specify NAT subnets, interface indices and session evaluation.	Design implemented
Implementation	Build telemetry, collectors, dashboard, datasets, controller, firmware and experiment runner.	Code implemented
Verification	Unit/native/integration/CLI/dashboard checks and original-ESP32 target build.	Software passes; hardware pending
Deployment	Generate private headers; flash boards; verify addresses and complete routing path.	Procedure ready; execution pending
Maintenance	Manage dependencies, records, secrets, regressions and recovery.	Ongoing; improvements proposed

The first increment established traffic capture and monitoring. The second corrected measurement semantics and introduced future-window evaluation. The third redesigned the dashboard and clarified missing observations. The fourth added NAT/SNMP firmware and gateway-aware integration. This increment removes demo artifacts and consolidates process evidence.

Security practices include local secrets, read-only management, restricted polling, bounded inputs and controlled writes. NIST SSDF provides guidance for organizing such practices throughout an SDLC [R4]. This report makes no formal SSDF compliance or independent security-certification claim.

11 Implementation structure and interfaces

Module	Responsibilities
server/server.py	Flask routes, traffic, reception, statistics and background probes
server/telemetry.py	Persistent events, migration, exports and delivery summaries
server/gateway_topology.py	Explicit NAT mode, source mapping and forwarded URLs
server/monitoring_status.py	Bounded collector reads and stale/failure states
server/templates and static	Dashboard, streaming, browsing and local Chart.js
monitoring	HTTP probes, SNMP collection/client and controller
datasets	Packet processing, causal windows, training and analysis
firmware/esp32_gateway	NAT/DHCP, counters, SNMP codec/service and configuration guard
esp32 code	Three sketches with boot identity, sequences and interval control
mininet and tests	Concurrent emulation and automated verification

Primary interfaces include POST /data, GET /api/dashboard, GET /api/telemetry/export, GET /api/delivery, GET /api/metrics/export and GET /api/export/sqlite. Event export supports run ID and timestamp filters. The dashboard reads actual snapshots rather than creating experiment records.

Gateway management is a separate UDP service. The Python client implements SNMPv2c; the Net-SNMP backend remains available for other agents. tools/configure_gateway.py generates private local headers from JSON and checks address separation. It writes matching network settings for the gateway and sensors.

The telemetry schema adds peer_ip to existing databases without deleting observations. Legacy packet analysis remains available; the old trainer requires an explicit legacy-heuristics option to identify its rule-reproduction purpose. Historical captures, reports and models are not silently replaced by the new workflow.

12 Verification and current evidence

Checks were rerun on 26 October 2026 after verified demo deletion. They establish tested software behavior, not physical throughput, radio reliability or model benefit.

Area	Checks	Current result
Measurement and pipeline	23 Python checks: delivery, failures, causal windows, CLI workflow, API and control	Passed
Gateway and native agent	13 Python checks, including eight calling the actual C codec	Passed
Dashboard status	Five Python checks for absent, current, stale, mismatched and partial samples	Passed
Presentation logic	Seven Node checks for polling, charts, missing states, labels and NAT mode	Passed
Gateway target build	Recorded original-ESP32 build with ESP-IDF 5.4.0	Compile/link succeeded
Desktop and mobile layout	Prior hidden desktop and 390-pixel mobile captures	Visually inspected; no page overflow or JS errors
Physical deployment	Live Wi-Fi forwarding, ACL, probes and actuation	Not validated
Real Mininet run	Concurrent workloads on Linux	Pending

Native tests cover all 19 monitored instances, 64-bit precision, GETNEXT order, SET rejection, malformed/truncated packets, bounded responses, 5,000 mutations and a loopback UDP exchange with the Python client. Identity and migration checks verify that two sensors remain distinct behind one translated address.

The recorded gateway image uses 707,718 bytes of its application partition and 32,856 bytes of static RAM. Runtime Wi-Fi, task and NAT heap use is additional. The checked unconfigured image stops before starting Wi-Fi; private headers and a rebuild are required before deployment.

Known warnings concern deprecated UTC datetime calls and an old example docstring escape sequence. Checks pass despite these warnings. Demo screenshots were visual test aids, not network results; their dummy records and images have been removed. Test sources and verification records remain the report evidence.

13 Deployment and operational process

1. Confirm four suitable boards, stable power and the correct serial port for each.

2. Keep the PC on upstream MiFi Wi-Fi and choose a different downstream /24. Check client isolation does not block PC/gateway access.

3. Create private JSON, generate ignored local headers and set addresses, passwords, community and optional control token.

4. Build and flash the dedicated gateway. Verify hotspot, upstream DHCP address and counter-installation messages.

5. Reflash the sensor sketches with generated downstream settings. Keep static sensors outside the phone DHCP pool.

6. Start gateway-mode Flask with a unique run ID and actual station address. Select the collector file/interface.

7. Start probes and collection using the same run ID. Check warmup, rates, source identities and forwarded RTT.

8. Begin observation-only operation. Enable intervention only after the complete measurement path is verified.

Example server environment

```
$env:IOT_RUN_ID = "gateway_pilot_01"
$env:IOT_USE_GATEWAY = "1"
$env:IOT_GATEWAY_UPLINK_IP = "192.168.8.110"
$env:IOT_SENSOR_SUBNET = "192.168.50.0/24"
$env:IOT_GATEWAY_IP = "192.168.8.110"
$env:IOT_GATEWAY_INTERFACE = "2"
$env:IOT_SNMP_CSV = "$PWD\datasets\snmp_ap.csv"
python server/server.py
```

Addresses are examples. Use values from the configured network. Do not commit credential headers or private JSON. Do not expose lab SNMPv2c to the Internet. The dashboard lacks a production authentication layer; external deployment would need an appropriate server, access controls and protected transport.

Recovery checks must include gateway reconnection, a changed DHCP address, collector failure, controller loss and sensor restart. Document restoration of the normal sensor interval. Confirm the whole measurement path before collecting final training or comparison data.

14 Experimental validation plan

Physical comparison

Use repeatable quiet, mixed-load and overload sessions. Fix sensor payloads, initial intervals, phone load, duration, placement and power. Record unique run IDs, addresses and firmware versions. Declare a warmup period and run several repetitions. Alternate or randomize strategy order to reduce environmental bias.

Compare no intervention, a fixed-threshold controller and the model controller. Measure sensor delivery, HTTP RTT, probe failures, update rate, gateway counters and application throughput together. Report tradeoffs if lower delay comes from reduced sensor updates or phone throughput. An altered workload alone does not establish a better predictive policy.

Reserve complete final test sessions before tuning. Use separate development sessions for threshold/model changes. Compare base and SNMP features on identical held-out rows. Report class metrics and confusion counts. Add event warning lead time, false alarms per hour and missed events; these event-level measures are proposed extensions, not current trainer outputs.

Controlled emulation

The Mininet baseline and optimized scenarios share bandwidth, delay and configured loss. Optimized changes offered UDP rate from 3 to 0.5 Mbit/s. A separate stream_cap scenario changes phone parallelism. All clients and ping probes start concurrently, with separate iperf server ports. Raw outputs and configuration are retained for each repetition.

This is a fixed-action comparison, not the learned controller. Per-link shaping and deliberate host differences are not a calibrated wireless airtime model. Failed commands retain missing metrics; confirmed ping no-reply results retain measured probe loss. Retransmissions, jitter and RTT retain distinct meanings.

Evidence required for a final claim

A final conclusion needs physical acceptance, held-out prediction results, repeated strategy comparisons, uncertainty across repetitions and intervention cost. Until then, the outcome is an implemented and software-verified prototype with a defined evaluation method. Do not derive numerical performance gains from synthetic demonstrations or historical rule-label accuracy.

15 Maintenance risks and project cleanup

Risk	Consequence	Response
ESP32 resource pressure	Gateway bottleneck or restart	Measure minimum heap, allocation failures, restart reason and forwarding capacity.
Unencrypted management	Community or lab data exposure	Restrict the lab, preserve collector ACL and consider protected management later.
NAT identity ambiguity	Phone aggregation or spoofed device IDs	Retain peer_ip, label groups and add authenticated identity if required.
Controller disappearance	Sensor remains rate-limited	Document recovery; add an expiring device-side lease in a future increment.
Leakage or workload bias	Overstated predictive/control benefit	Hold out sessions, match comparisons and retain raw provenance.
Dependency/API changes	Build or behavior regression	Keep the pinned target and test dependency updates before release.

Maintain a change record linking requirements, code, tests and deployment outcomes. Version firmware and models with configuration. Add retention limits and backups for growing data. Remove demonstrations after use without treating real observations or research references as disposable caches.

Cleanup performed on 26 October 2026

Category	Files removed	Verification
Isolated demo data and logs	7	Database contained only DEMO_synthetic_traffic
Dummy images and metadata	7	Capture metadata explicitly marked demo_only
Demo/capture launchers	2	Dedicated synthetic-data scripts
Redundant Python installers	3	Working extracted runtimes and pip verified

The matching hidden demo process was stopped and the two empty demo folders removed. Original captures, datasets, models, source, build tools and research documents remain. Backups, legacy references and broader caches were retained because their retention value was not established. The machine-readable ledger is reports/cleanup_manifest.json.

16 Requirements traceability matrix

The matrix links requirements to implementation and current verification. A board acceptance check is not replaced by a software test.

Req	Implementation	Verification	State
FR01	Three sensor sketches	Boot/sequence design and live transmission gate	Hardware pending
FR02	gateway.c; gateway_config.c	Recorded target build and route acceptance gate	Build verified
FR03	snmp_codec.c; snmp_service.c	Native GET/walk/rejection and UDP exchange tests	Software verified
FR04	snmp_collect.py; snmp_wire.py	Counter and native client interoperability checks	Software verified
FR05	telemetry.py; gateway_topology.py	NAT HTTP identity and migration checks	Software verified
FR06	Telemetry delivery summary	Duplicate, late-arrival and boot separation tests	Software verified
FR07	probe.py; forwarded probes	Target mapping and outcome windows; live gate	Live check pending
FR08	Dashboard template, JS and CSS	Seven Node checks, five status checks and visual review	Software verified
FR09	Flask export routes	HTTP/export and template/static integration tests	Software verified
FR10	build_windows.py	Future labels, missing windows and SNMP alignment	Software verified
FR11	train_windows.py	Held-out split, imputation and CLI workflow tests	Software verified
FR12	control.py; sensor /config	Mocked action and bounds; physical actuation gate	Hardware pending
FR13	Detailed experiment runner	Concurrent launch and matched configuration checks	Linux run pending
FR14	configure_gateway.py	Matching headers, subnet/community validation	Software verified

Release gate

The pilot is complete only when four configured boards operate, the server sees distinct sensor identities, forwarded probes reach every sensor, counters respond to actual load and the controller changes and restores the selected interval. Record those checks as run evidence. A final research release also requires the held-out and comparative experiments in Section 14.

17 References and terminology

Project evidence

[P1] IMPLEMENTATION_GUIDE.md and firmware/esp32_gateway/README.md. Operating and configuration instructions.

[P2] Source in server, monitoring, datasets, firmware/esp32_gateway, esp32 code and mininet. Inspected for this report.

[P3] tests/test_measurement_pipeline.py, tests/test_gateway_snmp.py, tests/test_dashboard_status.py and tests/dashboard.test.cjs. Verification sources; 41 Python/native tests and seven Node checks passed on 26 October 2026.

[P4] VALIDATION.md. Recorded successful gateway build, native checks and physical-validation limitations. The target build was not repeated solely to prepare this report.

[P5] reports/cleanup_manifest.json. Verified demo and installer deletion ledger. Prior dummy screenshots were interface demonstrations, not research evidence, and have been deleted.

External references

[R1] Espressif Systems. ESP32 Wi-Fi API reference. AP/station single-channel behavior. Accessed 26 October 2026.

https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html

[R2] Espressif Systems. AP and station NAT example, version 5.4.2. API design reference; the project builds against pinned 5.4.0. Accessed 26 October 2026.

https://github.com/espressif/esp-idf/tree/v5.4.2/examples/wifi/softap_sta

[R3] scikit-learn developers. Common pitfalls and recommended practices. Training-only preprocessing and leakage prevention. Accessed 26 October 2026.

https://scikit-learn.org/stable/common_pitfalls.html

[R4] NIST. SP 800-218 Secure Software Development Framework Version 1.1. Security guidance across the SDLC; no compliance claim. Accessed 26 October 2026.

https://csrc.nist.gov/pubs/sp/800/218/final

Terminology

SDLC means software development lifecycle. SNMP is Simple Network Management Protocol. A MIB defines management objects and meanings. NAT translates addresses; NAPT also translates ports. RTT is round-trip time. A run ID identifies one experiment session. Software-interface counters observe the lwIP/driver boundary, not the RF channel directly.